

# IDP: RPi4 projects

- [Prerequisites](#)
- [UART support](#)
- [SPI Support](#)
- [Storage Connection](#)
  - [SPI Flash](#)
  - [SPI SD](#)
  - [Internal SD](#)
- [Ethernet Support](#)
  - [demo\\_iot\\_app](#)
- [Multicore Support](#)
  - [Multicore Support for the RPi4](#)
  - [Multicore Application](#)

## Prerequisites

- Rule of thumb: checkout branch *rpi4\_porting*

### Checkout correct submodules

```
cd seos_tests
git checkout rpi4_porting
git submodule update --recursive --init
```

- when building an application for the RPi4, it is necessary to put *-DAARCH64=1* in the build instructions in order to build for 64-bit architecture

## UART support

- *demo\_hello\_world* is used to verify basic support
- as it prints a string on the screen, it also verifies UART support

### Build demo\_hello\_world

```
seos_tests/seos_sandbox/scripts/open_trentos_build_env.sh seos_tests/seos_sandbox/build-system.sh seos_tests
/src/demos/demo_hello_world rpi4 build-rpi4-Debug-demo_hello_world -DCMAKE_BUILD_TYPE=Debug -DAARCH64=1
```

## SPI Support

- Driver: *RPi\_SPI\_LED* (branch: *rpi4\_porting*)
- *demo\_spi\_led* is used to verify SPI support

### SPI LED demo

```
seos_tests/seos_sandbox/scripts/open_trentos_build_env.sh seos_tests/seos_sandbox/build-system.sh seos_tests
/src/demos/demo_spi_led rpi4 build-rpi4-Debug-demo_spi_led -DCMAKE_BUILD_TYPE=Debug -DAARCH64=1
```

## Storage Connection

The following storages were connected and tested on the RPi4:

- W25Q64 NOR flash connected via SPI adapter
- SD card connected via SPI adapter
- SD card via internal SD card slot

### SPI Flash

- Driver: *RPi\_SPI\_Flash* (branch: *rpi4\_porting*)
- *test\_storage* is used to verify SPI flash support

### SPI Flash test

```
seos_tests/seos_sandbox/scripts/open_trentos_build_env.sh seos_tests/seos_sandbox/build-system.sh seos_tests/src/tests/test_storage rpi4 build-rpi4-Debug-test_storage -DCMAKE_BUILD_TYPE=Debug -DAARCH64=1
```

## SPI SD

- Driver: *RPi\_SPI\_SD* (branch: *rpi4\_porting*)
- *test\_storage\_interface* is used to verify SPI SD support

### SPI SD demo

```
seos_tests/seos_sandbox/scripts/open_trentos_build_env.sh seos_tests/seos_sandbox/build-system.sh seos_tests/src/tests/test_storage_interface rpi4 build-rpi4-Debug-test_storage_interface_sd -DCMAKE_BUILD_TYPE=Debug -DAARCH64=1
```

## Internal SD

- Driver: *RPi\_SD* (branch: *rpi4\_porting*)
- *test\_storage\_interface* is used to verify internal SD support

### Internal SD demo

```
seos_tests/seos_sandbox/scripts/open_trentos_build_env.sh seos_tests/seos_sandbox/build-system.sh seos_tests/src/tests/test_storage_interface rpi4 build-rpi4-Debug-test_storage_interface_sd -DCMAKE_BUILD_TYPE=Debug -DAARCH64=1
```

Note: Both the SPI SD driver (*RPi\_SPI\_SD*) and the internal SD driver (*RPi\_SD*) are tested with *test\_storage\_interface* on branch *rpi4\_porting*.

## Ethernet Support

- Driver: *NIC\_RPi* (branch: *rpi4\_porting*)
- *demo\_raspi\_ethernet* verifies basic ethernet support
- *demo\_iot\_app\_rpi3* (branch: *rpi4\_porting*) integrates ethernet driver into bigger context
- configure the U-Boot environment accordingly (interrupt the boot process to fall back to U-Boot command line):

- **configure u-boot environment**

```
u-boot> setenv serverip 10.0.0.1
u-boot> setenv ipaddr 10.0.0.11
u-boot> setenv netmask 255.255.255.0
u-boot> saveenv
u-boot> boot
```

### demo\_raspi\_ethernet

```
seos_tests/seos_sandbox/scripts/open_trentos_build_env.sh seos_tests/seos_sandbox/build-system.sh seos_tests/src/demos/demo_raspi_ethernet rpi4 build-rpi4-Debug-demo_raspi_ethernet -DCMAKE_BUILD_TYPE=Debug -DAARCH64=1
```

## demo\_iot\_app

1. Build the demo application:

#### demo\_iot\_app

```
seos_tests/seos_sandbox/scripts/open_trentos_build_env.sh seos_tests/seos_sandbox/build-system.sh  
seos_tests/src/demos/demo_iot_app_rpi3 rpi4 build-rpi4-Debug-demo_iot_app_rpi4 -  
DCMAKE_BUILD_TYPE=Debug -DAARCH64=1
```

2. Create *BOOT*, *CONFIGSRV* and *LOG* partition with *prepare\_sd\_card.sh* script:

#### partition SD card

```
sudo ./prepare_sd_card.sh /dev/<mount-point>
```

3. Copy the RPi4 boot files and the application image *os\_image.elf* to the *BOOT* partition of the SD card.
4. Generate the configuration provisioned binary files (*BLOB.bin*, *DOMAIN.bin*, *PARAM.bin*, *STRING.bin*):

#### generate configuration provisioned binary files

```
seos_tests/seos_sandbox/tools/cpt/build_cpt/cpt -i seos_tests/src/demos/demo_iot_app_rpi3  
/configuration/config.xml
```

5. Copy these files to the *CONFIGSRV* partition of the SD card
6. Start the mosquito server application:

#### start mosquito server

```
docker run -it -p 8883:8883 -v /home/lukas/Desktop/seos_tests/src/demos/demo_iot_app_rpi3  
/mosquitto_configuration:/mosquitto/config/ eclipse-mosquitto
```

7. Configure the Ethernet interface of the laptop to ip address *10.0.0.1*
8. Connect RPi4 with Ethernet cable to laptop
9. Power on the RPi4
10. Interrupt booting and jump into U-Boot command line
11. Enter the following information in the U-Boot command line:

#### configure u-boot environment

```
u-boot> setenv serverip 10.0.0.1  
u-boot> setenv ipaddr 10.0.0.2  
u-boot> setenv netmask 255.255.255.0  
u-boot> saveenv  
u-boot> boot
```

## Multicore Support

The multicore support part can be split into two separate subtasks:

- investigate on multicore support for the RPi4 on the seL4 microkernel
- create multicore application

### Multicore Support for the RPi4

The following branches were created:

- SMP driver in repository *sel4\_tools* (branch: *multicore\_support*): *sdk-sel4-camkes/tool/sel4*
- necessary modifications to the *sel4* kernel repository (branch: *multicore\_support*): *sdk-sel4-camkes/sel4*

Unfortunately, the startup code for the secondary cores is not working completely. To start up the secondary cores properly, follow reference implementations from Fiasco.OC and the *circle* library. More precise information about the implementation and findings can be found in the TUM Gitlab Wiki.

## Multicore Application

The multicore application is located at `seos_systems/demo_multicore`. The application runs an approximation of Pi via the Monte-Carlo method on up to 4 different CAmkES instances. The application can be configured during build time with the following configuration parameters:

- execution modes:
  - same core, same priority
  - same core, different priorities
  - different cores, same priority
  - different cores, different priorities
- number of CAmkES instances
- number of iterations for Monte Carlo method

The application was test on the Sabre Lite and behaves as expected.